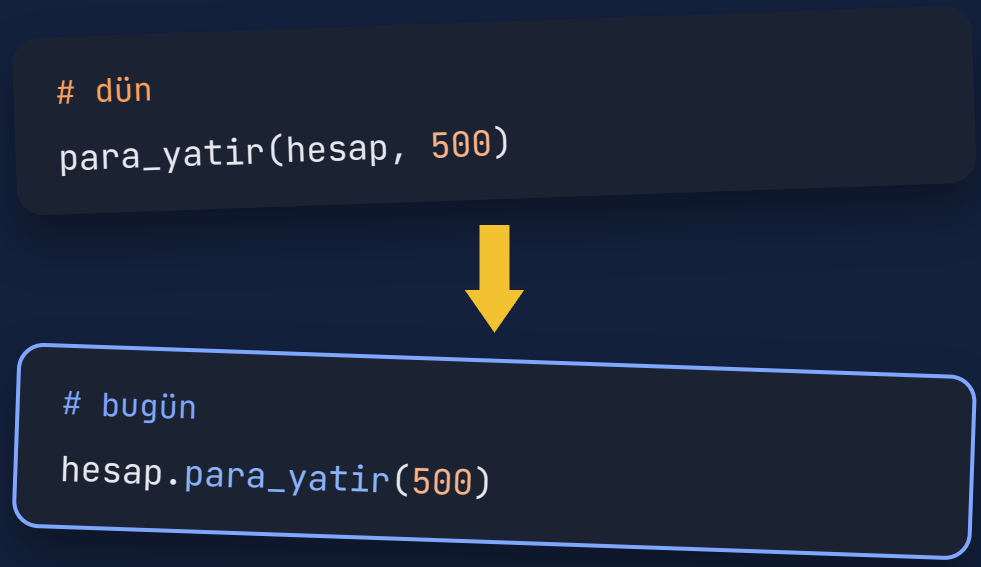
NESNE YÖNELİMLİ PROGRAMLAMA-1

Programlama Paradigmaları

Prosedürelden nesne yönelimliye geçiş

prosedürel fonksiyonel nesne yönelimli



Erhan AKAGÜNDÜZ

BU HAFTA

Dersin sonunda neler yapabileceksiniz?

Paradigma

“Programlama paradigması” kavramını açıklamak ve başlıca türlerini saymak

Prosedürel

Prosedürel kodun güçlü yanlarını ve büyüdükçe ortaya çıkan sorunlarını görmek

Nesne yönelimli

Sınıf, nesne, özellik ve metot kavramlarını ayırt etmek; dört ilkeyi tanımak

Dönüşüm

Prosedürel bir programı isim–fiil analiziyle sınıflara dönüştürmek

Bugün sözdizimi değil, düşünme biçimi öğreniyoruz. Sınıfların ayrıntılı yazımı (**__init__**, **self**) 3. haftanın konusu; burada önizleme olarak göreceğiz.

GEÇEN HAFTADAN BUGÜNE

Bu kodu yazabiliyoruz. Peki sorun ne?

hafta1_ogrenci.py

```python
ogrenci = {"ad": "Ada", "notlar": [90, 70]}
def not_ekle(o, n):
    o["notlar"].append(n)

def ortalama(o):
    return sum(o["notlar"]) / len(o["notlar"])

not_ekle(ogrenci, 80)
print(ogrenci["ad"], ortalama(ogrenci))
```

▷ ÇIKTI

```
Ada 80.0
```

Kod çalışıyor. Ama şu sorulara bir bakın:

Veri (**ogrenci**) ve onu işleyen fonksiyonlar neden **ayrı** yerlerde?

Her fonksiyon neden öğrenciyi **parametre** olarak istiyor?

Biri **"notlar"** yerine **"not"** yazarsa ne olur?

KAVRAM

Paradigma: nasıl düşündüğün

Programlama paradigması, programı nasıl düzenleyeceğimizi belirleyen **yaklaşım ve kurallar bütünüdür**. Dil değil, bakış açısıdır.

Aynı hedef: **akşam yemeği hazırlamak**. Üç farklı düşünme biçimi:

Tarif kitabı

"Suyu kaynat, makarnayı at, 8 dakika bekle..." **Adım adım ne yapılacağı.**

→ Prosedürel

Restoranda sipariş

"Bir makarna, az tuzlu." **Ne istediğini söylersin**, nasıl yapıldığıyla ilgilenmezsin.

→ Bildirimsel

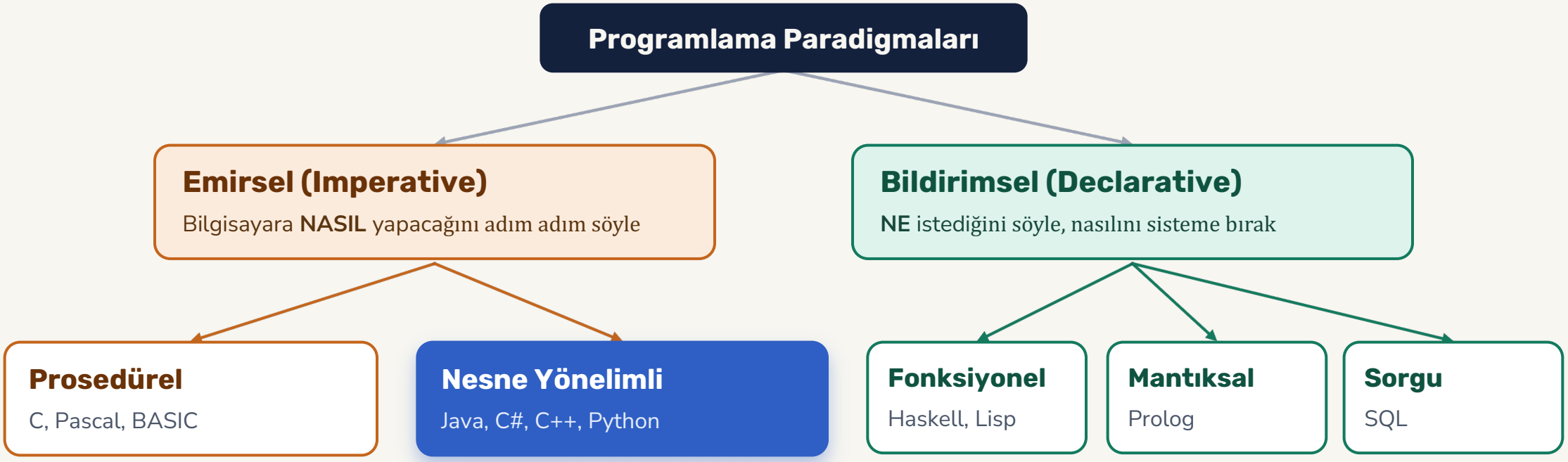
Mutfak ekibi

Aşçı, garson, kasiyer... **Her biri kendi bilgisini ve işini taşır**, birbirine mesaj gönderir.

→ Nesne yönelimli

PARADİGMA HARİTASI

İki büyük aile: “nasıl” ve “ne”



☆ **Python çok paradigmalıdır:** prosedürel, nesne yönelimli ve fonksiyonel tarzların hepsini destekler.

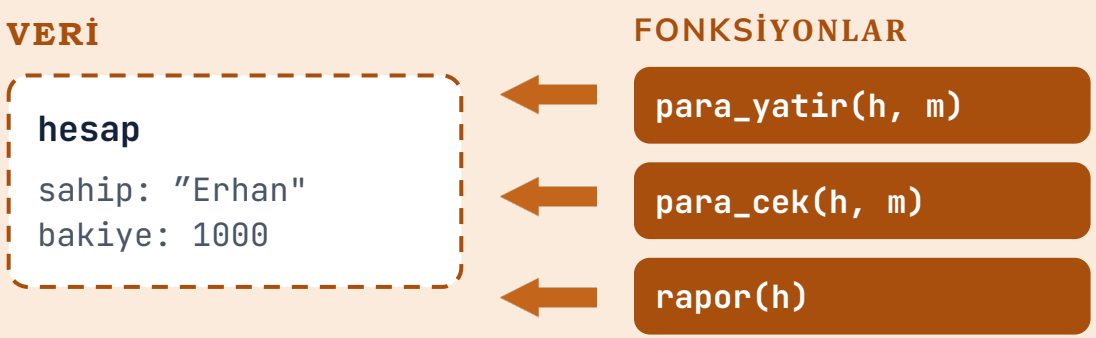
BÖLÜM 1

01 Prosedürel

Veri bir yanda, onu işleyen fonksiyonlar diğer yanda. Program, yukarıdan aşağı akan adımlar dizisi.

tarif mantığı · banka hesabı örneği · güçlü yanlar · üç büyük sorun

Veri pasif bekler, fonksiyonlar onu işler



Kendi başına hiçbir şey yapamaz.

Prosedürel programlama, programı birbirini çağıran **prosedürler** (fonksiyonlar) ve bunların üzerinde çalıştığı **veri** olarak düzenler.

- Odak: **yapılacak işler** (fiiller)
- Veri genelde ortak, fonksiyonlara parametreyle gider
- Kod yukarıdan aşağı, adım adım akar

📖 Tarif kitabı gibi: malzemeler tezgâhta durur, tarifin adımları onları sırayla işler.

Banka hesabı, prosedürel tarzda

banka_prosedurel.py

```python
hesap = {"sahip": "Ayşe", "bakiye": 1000}
def para_yatir(h, miktar):
    h["bakiye"] += miktar
def para_cek(h, miktar):
    if miktar > h["bakiye"]:
        print("Yetersiz bakiye!")
        return
    h["bakiye"] -= miktar
def rapor(h):
    print(f"{h['sahip']}: {h['bakiye']} TL")

para_yatir(hesap, 500)
para_cek(hesap, 2000)
para_cek(hesap, 300)
rapor(hesap)
```

▷ ÇIKTI

```
Yetersiz bakiye!
Ayşe: 1200 TL
```

Çağrı	bakiye
başlangıç	1000
para_yatir(500)	1500
para_cek(2000)	1500 (reddedildi)
para_cek(300)	1200

Her fonksiyon **h** parametresiyle “hangi hesap?” diye sormak zorunda.

PROSEDÜREL · GÜÇLÜ YANLAR

Prosedürel kötü değildir

Basit

Öğrenmesi kolay; kod yazıldığı sırayla okunur.

Hızlı başlangıç

Küçük betikler için tasarım yükü yoktur.

Yeniden kullanım

Fonksiyonlar tekrar eden işi tek yerde toplar.

Performans

Donanıma yakın, öngörülebilir çalışma (ör. C dili).

Bugün nerede kullanılıyor?

otomasyon betikleri · işletim sistemi çekirdekleri · gömülü sistemler · tek seferlik veri analizleri

Sorun, program büyüdüğünde başlar.

SORUN 1 · KORUMASIZ VERİ

Kurallar fonksiyonda, ama veri herkese açık

sorun1.py

```
hesap = {"sahip": "Erhan", "bakiye": 1000}

# para_cek() kontrolünü atlayan biri:
hesap["bakiye"] -= 5000
hesap["bakyie"] = 99      # yazım hatası!

print(hesap)
```

▷ ÇIKTI

```
{'sahip': 'Erhan', 'bakiye': -4000, 'bakyie': 99}
```

Ne ters gitti?

- ⚠ Bakiye **−4000** oldu; “yetersiz bakiye” kuralı hiç çalışmadı.
- ⚠ Yazım hatası hata vermedi, **yeni bir anahtar** yarattı.
- ⚠ Bunu kodun **herhangi bir yerinde**, herhangi biri yapabilir.

🔒 OOP’nin cevabı: veriyi nesnenin içine koy, erişimi metotlarla sınırla → **kapsülleme**.

SORUN 2 · BÜYÜYEN if/elif ZİNCİRLERİ

Her yeni tür, her fonksiyonda yeni bir elif

sorun2.py

```python
def faiz_hesapla(h):
    if h["tur"] == "vadesiz":
        return 0
    elif h["tur"] == "vadeli":
        return h["bakiye"] * 0.40
    elif h["tur"] == "altin":
        return h["bakiye"] * 0.05
    # yeni tür = buraya yeni elif

h1 = {"tur": "vadeli", "bakiye": 1000}
h2 = {"tur": "altin", "bakiye": 1000}
print(faiz_hesapla(h1), faiz_hesapla(h2))
```

▷ ÇIKTI `400.0 50.0`

“Döviz hesabı” eklenecek…

Türe göre davranan **her** fonksiyonu bul ve değiştir:

`faiz_hesapla` `komisyon` `rapor` `limit` `vergi`

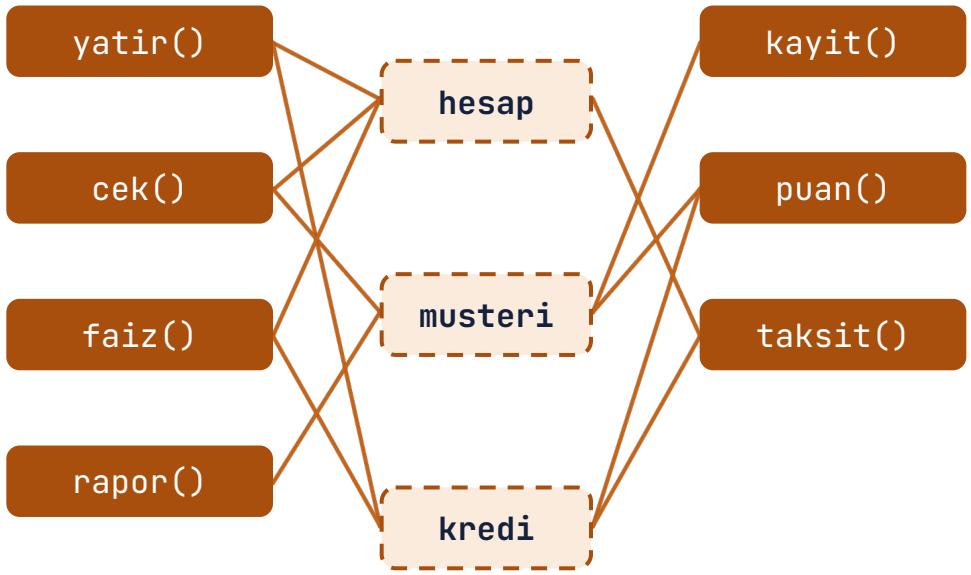
Birini unutursan, hata ancak çalışırken ortaya çıkar.

OOP’nin cevabı: her tür kendi kuralını kendi taşır → **kalıtım** ve **çok biçimlilik**.

SORUN 3 · ÖLÇEK

Spagetti mi, kapsüller mi?

PROSEDÜREL: HER FONKSİYON HER VERİYE UZANIR



OOP: HER VERİ KENDİ İŞLEMLERİYLE BİRLİKTE

Hesap

`yatir()` `cek()` `faiz()`

Musteri

`kayit()` `puan()` `rapor()`

Kredi

`taksit()` `onay()`

BÖLÜM 2 · KISA BİR GEZİNTİ

02

Diğer paradigmalar

Fonksiyonel ve bildirimsel düşünce: “nasıl” yerine “ne”.

saf fonksiyonlar | map / filter / lambda | aynı iş, üç tarz

Veriyi değiştirme, dönüştür

fonksiyonel.py

```
fiyatlar = [120, 45, 300, 80, 15]

kdvli = list(map(lambda f: f * 1.2, fiyatlar))
pahali = list(filter(lambda f: f > 100, fiyatlar))
toplam = sum(fiyatlar)
print(kdvli)
print(pahali)
print(toplam)
```

▷ ÇIKTI

```
[144.0, 54.0, 360.0, 96.0, 18.0]
[120, 300]
560
```

Saf fonksiyon

Aynı girdi → hep aynı çıktı. Dışarıdaki hiçbir şeyi değiştirmez.

Değişmez veri

fiyatlar hiç değişmedi; her adım yeni bir liste üretti.

Fonksiyon = değer

lambda ile isimsiz fonksiyon yazıp başka fonksiyona verebiliriz.

Veri analizi ve paralel işlemede çok güçlü.

KARŞILAŞTIRMA

Aynı iş, üç düşünme biçimi

cift_kareler.py

```python
sayilar = [1, 2, 3, 4, 5, 6]

# 1) Emirsel (adım adım)
sonuc = []
for s in sayilar:
    if s % 2 == 0:
        sonuc.append(s * s)
print(sonuc)

# 2) Bildirimsel (ne istediğini söyle)
print([s * s for s in sayilar if s % 2 == 0])

# 3) Fonksiyonel
print(list(map(lambda s: s * s,
               filter(lambda s: s % 2 == 0, sayilar))))
```

▷ ÇIKTI

```
[4, 16, 36]
[4, 16, 36]
[4, 16, 36]
```

1. Bilgisayara her adımı **sen** söylüyorsun.
2. Sonucun **neye benzediğini** tarif ediyorsun.
3. Fonksiyonları **zincir gibi** birbirine bağlıyorsun.

Hepsi doğru. Seçim; **okunabilirlik**, ekip ve problemin yapısına bağlı.

BÖLÜM 3

03

Nesne Yönelimli Programlama

Veri ve onu işleyen davranış, aynı kutuda: **nesne**.

- nesne = özellik + davranış
- sınıf ve nesne
- ilk sınıfımız
- dört temel ilke

Veriye dışarıdan uzanma, ona mesaj gönder

Nesne yönelimli programlama, programı kendi **verisini** ve **davranışını** taşıyan nesnelerin birbiriyle iletişimi olarak düzenler.

Prosedürel	`para_yatir(hesap, 500)`	"**Fonksiyon**, şu hesaba 500 ekle." Odak: **fiil**
Nesne yönelimli	`hesap.para_yatir(500)`	"**Hesap**, kendine 500 yatır." Odak: **isim** (nesne)

- Veri, sahibinin yanında ve korunaklı
- "Hangi hesap?" diye sormaya gerek yok
- Kod, gerçek dünyaya benzer

Her nesne iki şey taşır: ne olduğu ve ne yaptığı

Araba (bir nesne)

ÖZELLİKLER

ne olduğu · veri

```
marka = "Togg"
renk = "mavi"
hiz = 0
yakit = 80
```

DAVRANIŞLAR

ne yaptığı · işlem

```
hizlan()
fren_yap()
korna_cal()
depo_doldur()
```

hizlan() çağrılınca **hiz** artar: davranış, nesnenin kendi verisini değiştirir.

Günlük dil	OOP terimi	Python'da
Özellik	Nitelik (attribute)	self.hiz
Davranış	Metot (method)	def hizlan(self)

Öğrenci

Özellik: ad, numara, notlar

Davranış: not_ekle(), ortalama()

Banka hesabı

Özellik: sahip, bakiye

Davranış: para_yatir(), para_cek()

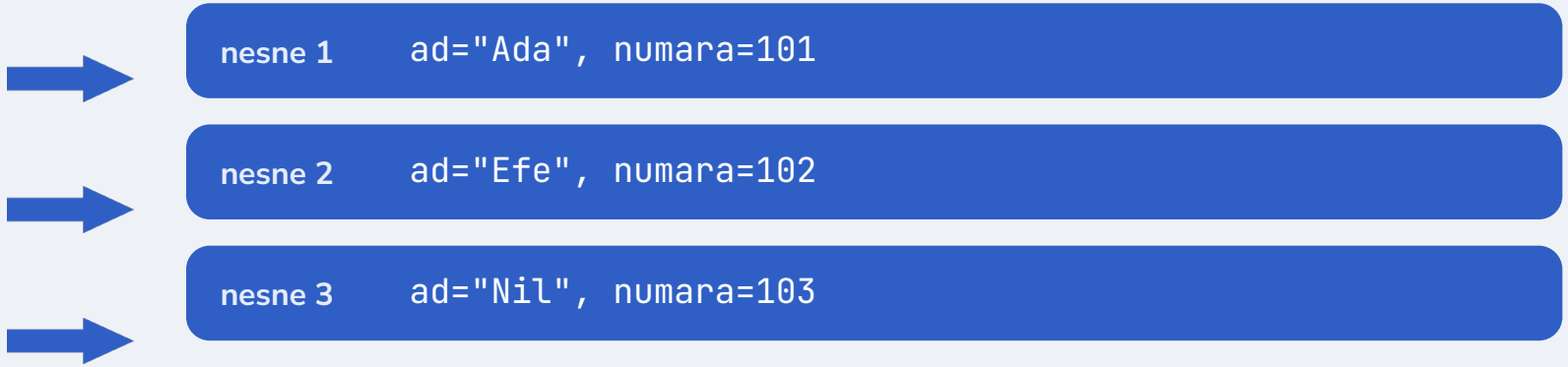
Sınıf kalıptır, nesne o kalıptan çıkan üründür

SINIF (class)

Ogrenci

```
ad = ?
numara = ?
notlar = ?
_________
not_ekle()
ortalama()
```

Plan: hangi özellik ve davranışlar olacak?



Aynı plan, **farklı veriler**. Her nesne kendi değerlerini taşır.

Mimari plan → o plana göre yapılmış evler

Nesneye sınıfın **örneği** (instance) de denir

Aynı banka hesabı, bu kez bir sınıf olarak

banka_oop.py

```python
class Hesap:
    def __init__(self, sahip, bakiye):
        self.sahip = sahip
        self.bakiye = bakiye
    def para_yatir(self, miktar):
        self.bakiye += miktar
    def para_cek(self, miktar):
        if miktar > self.bakiye:
            print("Yetersiz bakiye!")
            return
        self.bakiye -= miktar
    def rapor(self):
        print(f"{self.sahip}: {self.bakiye} TL")
```

```python
# kullanım
h = Hesap("Erhan", 1000)
h.para_yatir(500)
h.para_cek(2000)
h.para_cek(300)
h.rapor()
```

▷ ÇIKTI

```
Yetersiz bakiye!
Erhan: 1200 TL
```

class yeni bir tür (plan) tanımlar

__init__ nesne doğarken çalışır

self "bu nesnenin kendisi" → 3. hafta

YAN YANA

Aynı işlem, iki yazım

Prosedürel	işlem	Nesne yönelimli
`hesap = {"sahip": "Erhan", "bakiye": 1000}`	oluşturma	`h = Hesap("Erhan", 1000)`
`para_yatir(hesap, 500)`	para yatır	`h.para_yatir(500)`
`para_cek(hesap, 300)`	para çek	`h.para_cek(300)`
`rapor(hesap)`	rapor	`h.rapor()`
`hesap["bakiye"]`	bakiyeyi oku	`h.bakiye`

Sonuç aynı: **Erhan: 1200 TL**. Fark, kodun **kim–ne yapıyor** sorusuna verdiği cevapta.

SÜRPRİZ

Python’da her şey bir nesnedir

hersey_nesne.py

```
print(type(5))
print(type("merhaba"))
print(type([1, 2]))

isim = "python"
print(isim.upper())        # metot!
liste = [3, 1, 2]
liste.sort()               # metot!
print(liste)
print((5).bit_length())
```

▷ ÇIKTI

```
<class 'int'>
<class 'str'>
<class 'list'>
PYTHON
[1, 2, 3]
3
```

Farkında olmadan OOP yapıyordunuz

int, **str**, **list** birer sınıf; **5**, **"python"** ve **[3, 1, 2]** bu sınıfların nesneleri. **.upper()**, **.sort()** ise metotları.

OOP · DÖRT TEMEL İLKE

Dönemin geri kalanı bu dört sütun üzerine

Nesne Yönelimli Programlama

Kapsülleme

Veriyi koru, erişimi metotlarla sınırla.

Hafta 5

Kalıtım

Var olan sınıfı genişleterek yenisini üret.

Hafta 9–10

Çok biçimlilik

Aynı mesaj, her nesnede kendine özgü davranış.

Hafta 11

Soyutlama

Ayrıntıyı gizle, sadece gerekeni göster.

Hafta 12

Temel: sınıf ve nesne · Hafta 3–4

Sonraki dört slaytta her ilkeye banka hesabı üzerinden **kısa bir önizleme** yapıyoruz.

İLKE 1 · KAPSÜLLEME (ÖNİZLEME)

Bakiyeye sadece kapıdan girilir

kapsulleme.py

```
class Hesap:
    def __init__(self, bakiye):
        self.__bakiye = bakiye    # gizli
    def para_cek(self, miktar):
        if miktar > self.__bakiye:
            return "Reddedildi"
        self.__bakiye -= miktar
        return "Onaylandı"
    def bakiye_gor(self):
        return self.__bakiye

h = Hesap(1000)
print(h.para_cek(5000))
print(h.bakiye_gor())
try: print(h.__bakiye)
except AttributeError: print("Doğrudan erişim yok!")
```

▷ ÇIKTI

```
Reddedildi
1000
Doğrudan erişim yok!
```

Sorun 1 çözüldü

Bakiye artık dışarıdan **−4000** yapılamaz. Değişiklik yalnızca kuralı uygulayan **para_cek()** üzerinden olur.

Çift alt çizgi (__) ismi gizler. Ayrıntılar ve getter/setter: **5. hafta**.

Yeni tür = var olanı al, farkı ekle

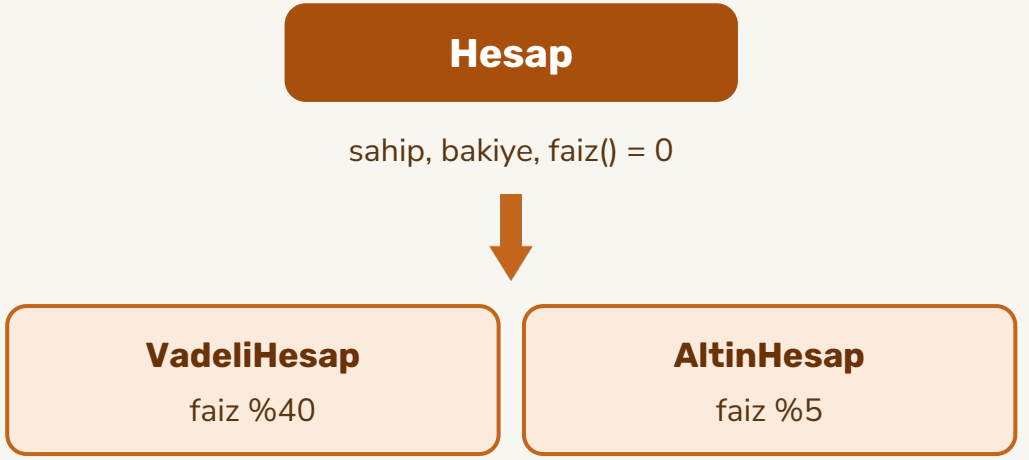
kalitim.py

```python
class Hesap:
    def __init__(self, sahip, bakiye):
        self.sahip = sahip
        self.bakiye = bakiye
    def faiz(self):
        return 0

class VadeliHesap(Hesap):
    def faiz(self):
        return self.bakiye * 0.40

v = VadeliHesap("Yamaç", 1000)
print(v.sahip)       # Hesap'tan miras
print(v.faiz())      # kendi kuralı
```

▷ ÇIKTI Yamaç · 400.0



Sorun 2 çözülüyor: yeni tür, yeni bir sınıftır. Eski fonksiyonlara elif eklemek gerekmez.

İLKE 3 · ÇOK BİÇİMLİLİK (ÖNİZLEME)

Aynı mesaj, farklı cevaplar

cok_bicim.py

```
class Kedi:
    def ses(self):
        return "Miyav"
class Kopek:
    def ses(self):
        return "Hav hav"
class Inek:
    def ses(self):
        return "Möö"

for hayvan in [Kedi(), Kopek(), Inek()]:
    print(hayvan.ses())
```

▷ ÇIKTI Miyav ↵ Hav hav ↵ Möö

TEK MESAJ: ses()

Kedi → "Miyav"

Kopek → "Hav hav"

Inek → "Möö"

Banka diliyle

```
for h in hesaplar:
    toplam += h.faiz()
```

if/elif yok: her hesap kendi faizini bilir.

Arabayı sürmek için motoru bilmen gerekmez

KULLANICININ GÖRDÜĞÜ

```
h.para_cek(300)
```

PERDE ARKASINDA OLANLAR

- bakiye kontrolü
- günlük limit
- komisyon hesabı
- işlem kaydı (log)
- güvenlik doğrulaması
- veritabanı güncelleme

Soyutlama: karmaşık ayrıntıları gizleyip kullanıcıya yalnızca **ne yapılabileceğini** göstermek.

- **Araba:** direksiyon, gaz, fren. Enjeksiyon sistemi gizli.
- **ATM:** “Para çek” düğmesi. Bankanın sistemi gizli.
- **Python: liste.sort()** nasıl sıralıyor? Bilmeden kullanıyoruz.

Soyut sınıflar ve arayüz tasarımı: **12. hafta**.

BÖLÜM 4

04

Geçiş

Prosedürel bir problemi nesne yönelimli tasarıma nasıl çeviririz?

isim–fiil analizi | önce / sonra | ne zaman hangisi? | mini proje

GEÇİŞ · YÖNTEM

İsimler sınıf olur, fiiller metot olur

PROBLEM TANIMI

Bir kütüphanede **üyeler kitap** ödünç alır ve iade eder. Her kitabın *adı*, *yazarı* ve *durumu* vardır. Üyenin *adı* ve ödünç aldığı *kitaplar* tutulur.

Kalın mavi = ana isim → sınıf *İtalik yeşil* = özellik → nitelik Altı çizili turuncu = fiil → metot

class Kitap

```
ad, yazar, durum
```

Metodu yok mu? *durum_degistir()* eklenebilir.

class Uye

```
ad, kitaplar
odunc_al(kitap)
iade_et(kitap)
```

3 ADIM

1. İsimleri bul → sınıf adayları
2. Sıfat ve "-in ..."ları bul → nitelikler
3. Fiilleri bul → metotlar

3. slayttaki öğrenci kodunu dönüştürelim

ÖNCE · PROSEDÜREL

```
ogrenci = {"ad": "Furkan", "notlar": [90, 70]}

def not_ekle(o, n):
    o["notlar"].append(n)

def ortalama(o):
    return sum(o["notlar"]) / len(o["notlar"])



not_ekle(ogrenci, 80)
print(ogrenci["ad"], ortalama(ogrenci))
```

SONRA · NESNE YÖNELİMLİ

```
class Ogrenci:
    def __init__(self, ad, notlar):
        self.ad = ad
        self.notlar = notlar
    def not_ekle(self, n):
        self.notlar.append(n)
    def ortalama(self):
        return sum(self.notlar) / len(self.notlar)

ogr = Ogrenci("Furkan", [90, 70])
ogr.not_ekle(80)
print(ogr.ad, ogr.ortalama())
```

▷ **İki çıktı da:** `Furkan 80.0`

✓ Sözlük → nitelikler, fonksiyonlar → metotlar, **o** parametresi → **self**

GEÇİŞ · KARŞILAŞTIRMA

Prosedürel ve nesne yönelimli, yan yana

Kriter	Prosedürel	Nesne yönelimli
Temel yapı taşı	Fonksiyon	Nesne (sınıftan üretilir)
Odak	Yapılacak işler (fiiller)	Varlıklar (isimler)
Veri ve davranış	Ayrı yerlerde	Aynı kapsülde
Veri güvenliği	Herkes doğrudan değiştirebilir	Kapsüllemeyle korunur
Yeni tür eklemek	Birçok fonksiyona elif eklenir	Yeni bir sınıf yazılır
Yeniden kullanım	Fonksiyonlar	Fonksiyonlar + kalıtım
En uygun olduğu yer	Küçük betikler, algoritmalar	Büyük, uzun ömürlü sistemler
Tipik dil	C	Java, C#

GEÇİŞ · KARAR

Ne zaman hangisi?

Prosedürel yeterlidir

- Tek seferlik betik, kısa otomasyon
- Girdi → işlem → çıktı şeklinde net bir akış
- Az sayıda veri türü, az kişi
- Bir matematik formülü, bir algoritma

```
dosyadaki satırları say ve yazdır
```

OOP'ye geçme zamanı

- Birbiriyle etkileşen birçok varlık var
- Veri ve kurallar birlikte korunmalı
- Aynı şeyin birçok türü var (vadeli, altın...)
- Kod yıllarca büyüyecek, ekip çalışacak

```
banka, e-ticaret, oyun, okul sistemi
```

⚠ **Aşırıya kaçmayın:** her şeyi sınıf yapmak da bir hatadır. Tek metotlu, verisiz bir sınıf çoğu zaman bir fonksiyon olmalıdır.

UYGULAMA · MİNİ PROJE

Alışveriş sepeti: iki sınıf birlikte çalışıyor

sepet.py

```
class Urun:
    def __init__(self, ad, fiyat):
        self.ad = ad
        self.fiyat = fiyat
class Sepet:
    def __init__(self):
        self.urunler = []
    def ekle(self, urun, adet=1):
        self.urunler.append((urun, adet))
    def toplam(self):
        return sum(u.fiyat * a for u, a in self.urunler)
    def fis(self):
        for u, a in self.urunler:
            print(f"{u.ad:<8} x{a}  {u.fiyat * a:>5} TL")
        print(f"{'TOPLAM':<12} {self.toplam():>5} TL")
```

```
# kullanım
s = Sepet()
s.ekle(Urun("Kalem", 15), 3)
s.ekle(Urun("Defter", 40))
s.ekle(Urun("Silgi", 10), 2)
s.fis()
```

▷ ÇIKTI

```
Kalem    x3     45 TL
Defter   x1     40 TL
Silgi    x2     20 TL
TOPLAM         105 TL
```

Sepet, **Urun** nesnelerini içinde taşıyor: nesneler birbirini kullanabilir.

KENDİNİ SINA

Tahmin et: ekranda ne görünür?

Soru 1 · Bağımsız nesneler

```
class Sayac:
    def __init__(self):
        self.deger = 0
    def artir(self):
        self.deger += 1

a = Sayac(); b = Sayac()
a.artir(); a.artir(); b.artir()
print(a.deger, b.deger)
```

2 1

Her nesnenin kendi **deger**'i var; a iki kez, b bir kez arttı.

Soru 2 · Durum değiştiren metot

```
class Lamba:
    def __init__(self):
        self.acik = False
    def degistir(self):
        self.acik = not self.acik

l = Lamba()
l.degistir(); l.degistir(); l.degistir()
print(l.acik)
```

True

False → True → False → True. Metot, nesnenin durumunu her çağrıda değiştirdi.

DİKKAT

Sık yapılan 6 yanılgı

“OOP her zaman daha iyidir.”

Hayır. Küçük betiklerde gereksiz karmaşıklık yaratır.

“Sınıf ile nesne aynı şey.”

Sınıf plandır; nesne o plandan üretilen, verisi olan örnektir.

“Paradigma = programlama dili.”

Paradigma düşünme biçimidir; bir dil birden çok paradigmayı destekler.

“Python’da prosedürel yazılmaz.”

Yazılır. 1. haftada yazdığımız her şey prosedüreldi.

“Metot sıradan bir fonksiyon.”

Metot bir nesneye bağlıdır ve o nesnenin verisiyle çalışır.

“Her isim bir sınıf olmalı.”

İsim–fiil analizi bir başlangıçtır; her adayı sınıf yapmak aşırı tasarımdır.

ÖZET

Bugün öğrendiklerimiz, dört cümlede

Paradigma

Programı düzenleme biçimidir. Emirsel “nasıl”ı, bildirimsel “ne”yi söyler.

Prosedürel

Veri ve fonksiyonlar ayrıdır. Basittir; ama büyüdükçe veri korumasız, kod dağınık kalır.

Nesne yönelimli

Nesne = nitelik + metot. Sınıf plandır. Dört ilke: kapsülleme, kalıtım, çok biçimlilik, soyutlama.

Geçiş

İsimler sınıf, sıfatlar nitelik, fiiller metot olur. Sözlük → nitelik, parametre → self.

“Ne yapılacak?” sorusundan “Kim yapacak?” sorusuna geçiyoruz.

PRATİK ÖDEV · GELECEK HAFTA

Önce prosedürel yaz, sonra dönüştür

1 · İsim–fiil analizi

"Sinemada müşteri seans seçer, koltuk ayırır ve bilet öder." Sınıf, nitelik ve metot listesini çıkar.

2 · Prosedürel kütüphane

Sözlük ve fonksiyonlarla kitap ekle, ödünç ver, iade al ve listele işlemlerini yaz.

3 · Dönüştür

2. ödevi **Kitap** ve **Uye** sınıflarına çevir. 20. ve 30. slayttaki kalıbı kullan.

4 · Karar ver

Kullandığın bir uygulamayı seç. Prosedürel mi, OOP mi? Gerekçeni 3–4 cümleyle yaz.

HAFTA 3

Sınıf ve nesne: __init__ ve self

Bugün önizlediğimiz sözdizimini satır satır açıyoruz:

```
def __init__(self, ad):
    self.ad = ad
```

self tam olarak kim?